Lance a Lance – Documentação do Projeto

Nov 4, 2026 · @Arthur Zanon de Andrade

Este documento descreve tudo o que o grupo precisa construir, medir e entregar até Nov 23, 2026 às 12h00: um treinador de xadrez em que um agente de IA explica os erros do jogador, ancorado em um motor de xadrez.

1. Visão geral

Problema. Motores como o Stockfish dizem qual lance era melhor, mas não por quê. Para iniciantes e intermediários, "+2,3" e uma sequência de lances dizem pouco, e um professor custa caro.

Objetivo. Analisar as partidas do jogador, explicar cada erro grave em português e indicar quais temas ele precisa treinar, sem que a IA invente lances.

Modalidade. Trabalho 2 de IA II, opção 3: integração com IA em contexto avançado (agente, MCP e RAG).

Escopo do MVP

- Importar partidas de um usuário do Chess.com ou colar um PGN.
- Analisar cada lance com o Stockfish e classificar imprecisões, erros e erros graves.
- Explicar cada erro em português, com o tema tático envolvido.
- Chat sobre a posição exibida no tabuleiro.
- Relatório de pontos fracos por tema e puzzles recomendados.

Fora do MVP (citados como próximos passos no pitch): jogar contra o motor com comentário ao vivo, comandos por voz, leitura do tabuleiro por foto e contas de usuário.

2. Requisitos

Oito requisitos funcionais são essenciais para o MVP; os demais entram se sobrar tempo depois de 18/11.

Requisitos funcionais

ID	Requisito	Prioridade
RF01	Importar as últimas N partidas (padrão 10) de um usuário do Chess.com, percorrendo os arquivos mensais do mais recente para o mais antigo	Essencial

ID	Requisito	Prioridade
RF02	Colar uma partida em PGN manualmente	Essencial
RF03	Analisar cada lance com o Stockfish e classificar em imprecisão, erro ou erro grave	Essencial
RF04	Exibir a partida em um tabuleiro navegável, com os lances coloridos pela classificação	Essencial
RF05	Explicar em português cada erro e erro grave, com tema tático e melhor lance	Essencial
RF06	Chat com o agente sobre a posição exibida no tabuleiro	Essencial
RF07	Relatório de pontos fracos por tema, somando várias partidas do jogador	Essencial
RF08	Recomendar puzzles da base da Lichess pelos temas fracos e pelo rating do jogador	Essencial
RF09	Mostrar a ficha de conceito (fonte do RAG) usada em cada explicação	Desejável
RF10	Gráfico da avaliação ao longo da partida	Desejável
RF11	Cache das análises e explicações já geradas	Desejável

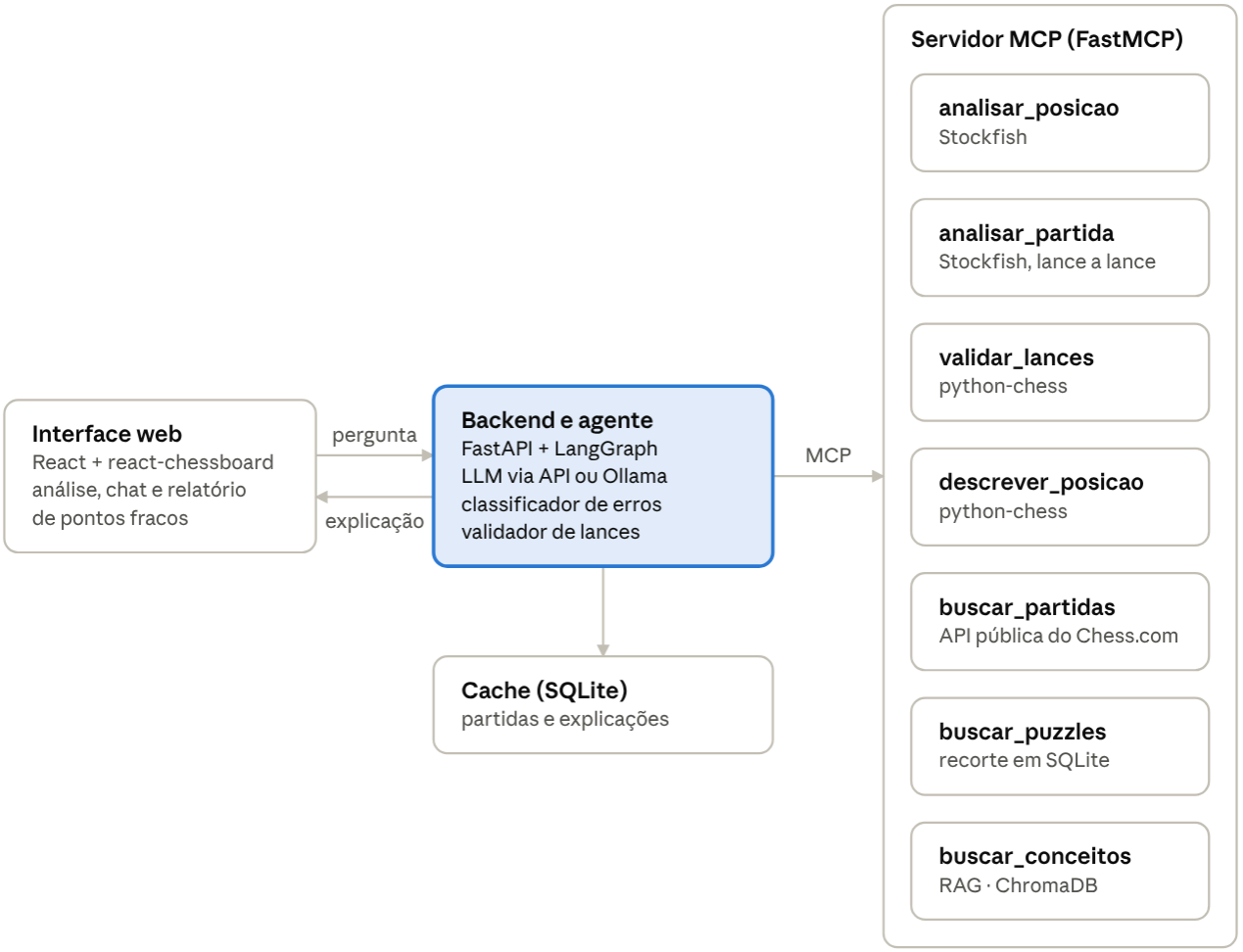
Requisitos não funcionais

- **RNF01 – Nenhum lance inventado:** todo lance citado pelo agente é validado com `python-chess` antes de chegar à tela.
- **RNF02 – Desempenho:** uma partida de 40 lances é analisada em até 60 segundos.
- **RNF03 – Usabilidade:** interface em português, utilizável em notebook e celular.
- **RNF04 – Reprodutibilidade:** o projeto sobe com um comando (`docker compose up`), configurado por um arquivo `.env`.
- **RNF05 – Segurança:** chaves de API nunca entram no repositório.
- **RNF06 – Rastreabilidade:** cada chamada de ferramenta do agente fica registrada em log, para depuração e para os prints do README.

3. Arquitetura

O sistema tem três camadas: interface web, backend com o agente de IA e um servidor MCP que expõe o motor, as regras do jogo e as bases de dados como ferramentas. O LLM nunca calcula lances; ele só interpreta o que as ferramentas devolvem.

O agente só afirma algo sobre o tabuleiro depois de consultar uma ferramenta



arquitetura · 3 camadas, 7 ferramentas MCP

A interface só conversa com o backend; o agente consulta as sete ferramentas via MCP e guarda tudo o que já calculou no cache.

Fluxo de análise de uma partida

1. O usuário informa um nome de usuário do Chess.com ou cola um PGN.
2. O backend busca as partidas (`buscar_partidas`) e as converte em uma lista de posições (FEN) com `python-chess` .
3. Para cada lance do jogador, `analisar_posicao` avalia a posição antes e depois do lance.
4. O backend calcula a queda na chance de vitória e classifica o lance (seção 8).

5. Para cada erro e erro grave, o agente reúne os fatos (`descrever_posicao` , linha do motor, ficha do RAG) e redige a explicação.
6. O validador confere todos os lances citados; se algum for inválido, a explicação é refeita.
7. O resultado é salvo em cache (SQLite) e exibido na interface.
8. Ao fim de várias partidas, os temas dos erros são somados para o relatório e a recomendação de puzzles.

4. Stack e dependências

Tudo em Python no backend e React no frontend, com bibliotecas prontas para as partes que não são o foco (regras, tabuleiro, gráficos). Fixem as versões em `requirements.txt` e `package-lock.json` já no primeiro dia.

Camada	Tecnologia	Para que serve
Linguagem	Python 3.11+	Backend, ferramentas e avaliação
Regras e PGN	`python-chess`	Ler PGN, gerar FEN, validar lances, falar com o motor (UCI)
Motor	Stockfish (binário oficial, versão mais recente)	Avaliação das posições e melhores lances
Servidor de ferramentas	FastMCP (`pip install fastmcp`)	Expor as ferramentas via MCP
Agente	LangGraph + `langchain-mcp-adapters`	Orquestrar o agente e carregar as ferramentas MCP
LLM	Um provedor via API (Claude, GPT ou Gemini) ou local via Ollama	Explicações e chat; definir um só no dia 1
Embeddings	Modelo multilíngue (ex.: `bge-m3` via `sentence-transformers`) ou o do provedor	Vetorizar as fichas de conceitos
Base vetorial	ChromaDB	Busca do RAG
API	FastAPI + Uvicorn	Endpoints consumidos pela interface
Cache e dados	SQLite + pandas	Análises salvas e recorte da base de puzzles

Camada	Tecnologia	Para que serve
Frontend	React + Vite + `react-chessboard` + `chess.js`	Tabuleiro, navegação pelos lances, chat
Gráficos	Recharts (front) e matplotlib (README)	Curva de avaliação e resultados da avaliação
Avaliação	RAGAS + scripts próprios	Métricas do RAG e das explicações
Testes	pytest	Ferramentas e classificador de erros
Infra	Docker Compose	Subir backend, MCP e frontend com um comando

5. Repositório e padrões de Git

Um único repositório no GitHub, com uma pasta por componente, para que o commit de cada integrante apareça na área dele.

```
lance-a-lance/
├── README.md
├── docker-compose.yml
├── .env.example              # LLM_API_KEY, STOCKFISH_PATH, ...
├── mcp_server/               # Integrante 1
│   ├── server.py             # FastMCP: registro das ferramentas
│   ├── tools/
│   │   ├── engine.py         # analisar_posicao
│   │   ├── board.py          # validar_lances, descrever_posicao
│   │   ├── chesscom.py       # buscar_partidas
│   │   ├── puzzles.py        # buscar_puzzles
│   │   └── concepts.py       # buscar_conceitos (RAG)
│   └── tests/
├── agent/                    # Integrante 2
│   ├── graph.py              # LangGraph: fluxo do agente
│   ├── classifier.py         # classificação de erros
│   ├── validator.py          # checagem dos lances citados
│   ├── prompts/
│   └── tests/
├── backend/                  # Integrante 1 (API) + 2 (agente)
│   ├── main.py               # FastAPI
│   └── db.py                 # SQLite (cache)
├── frontend/                 # Milton
│   └── src/
```

```
├── knowledge_base/           # fichas de conceitos (Markdown)
├── data/
│   ├── scripts/              # download e recorte da base de puzzles
│   └── README.md             # a base completa NÃO vai para o Git
├── evaluation/               # Integrante 2
│   ├── dataset.jsonl         # posições de teste
│   ├── run_eval.py
│   └── results/              # CSVs e gráficos gerados
└── docs/
    └── img/                  # prints usados no README
```

Padrões

- **Branches:** `main` sempre funcionando; cada tarefa em `feat/<assunto>` ou `fix/<assunto>`.
- **Pull requests:** todo merge na `main` passa por PR revisado por outro integrante. Isso gera evidência de contribuição de todos.
- **Commits:** pequenos e frequentes, no formato `tipo: descrição` (`feat: ferramenta validar_lances`, `fix: classificação de mate`, `docs: seção de avaliação no README`).
- **Segredos:** `.env` no `.gitignore`; só o `.env.example` vai para o repositório.
- **Arquivos grandes:** a base de puzzles e binários do Stockfish nunca entram no Git; o script em `data/scripts/` baixa tudo.

6. Fontes de dados

Todas as fontes são gratuitas e públicas; só a base de conceitos do RAG é escrita pelo grupo.

Fonte	O que fornece	Acesso	Licença
API pública do Chess.com	Partidas concluídas de qualquer jogador com perfil público, em JSON com o PGN de cada partida	`GET https://api.chess.com/pub/player/{usuario}/games/archives` lista os meses disponíveis; `GET .../player/{usuario}/games/{AAAA}/{MM}` traz as partidas do mês; sem chave de API	Dados públicos, somente leitura
Base de puzzles da Lichess	Cerca de 6 milhões de puzzles com FEN, solução, rating e temas	CSV compactado `lichess_db_puzzle.csv.zst` (colunas `PuzzleId, FEN, Moves, Rating, RatingDeviation, Popularity, NbPlays, Themes, GameUrl, OpeningTags`)	CC0 (domínio público)

Fonte	O que fornece	Acesso	Licença
Stockfish	Avaliação de posições e melhores lances	Binário local acionado pelo `python-chess` via UCI	GPL (usado como programa externo)
Base de conceitos	Fichas sobre temas táticos e estratégicos em português	Arquivos Markdown em `knowledge_base/`, escritos pelo grupo	Do grupo

Cuidados

- **Base de puzzles:** o arquivo completo tem gigabytes. O script de `data/` deve baixar, descompactar e salvar só um recorte (por exemplo, 50 mil puzzles com rating entre 600 e 2200) em SQLite.
- **API do Chess.com:** é pública e não pede chave, mas exige um cabeçalho User-Agent com o nome do app e um contato (sem ele, a resposta é erro 403). Fazer uma requisição por vez, guardar em cache os meses já baixados e lembrar que partidas recentes podem levar de meio dia a um dia para aparecer. A partida da demo deve ter sido jogada pelo menos na véspera.
- **Temas:** a coluna `Themes` usa nomes em inglês (`fork`, `pin`, `skewer`, `hangingPiece`, `discoveredAttack` ...). Criar um dicionário para o português, usado também pelo agente.
- **Base de conceitos:** cerca de 30 fichas, uma por tema, com definição, como reconhecer, como evitar e um exemplo em FEN. Escrever com as próprias palavras, citando a referência consultada no fim de cada ficha.

7. Componente 1 – Servidor MCP

Sete ferramentas determinísticas, todas devolvendo JSON. Elas são a única fonte de fatos sobre o tabuleiro que o agente pode usar.

Ferramenta	Entrada	Saída	Regras
`analisar_posicao`	`fen`, `profundidade` (padrão 18), `multipv` (padrão 3)	Avaliação (centipeões ou mate), 3 melhores lances em SAN, linha principal	Limite de tempo por posição; uma instância do motor reaproveitada

Ferramenta	Entrada	Saída	Regras
`analisar_partida`	`pgn`, `cor_do_jogador`	Lista por lance: avaliação antes e depois, queda de chance de vitória, classificação, melhor lance	Usa `analisar_posicao`; resultado vai para o cache
`validar_lances`	`fen`, `lances` (lista em SAN ou UCI)	Para cada lance: legal ou não, SAN normalizado	Base do validador do agente
`descrever_posicao`	`fen`	Material, peças atacadas e indefesas, cravadas, xeques possíveis, rei exposto	Só fatos calculados com `python-chess`, sem opinião
`buscar_partidas`	`usuario`, `max` (padrão 10), `ritmo` (opcional)	Lista de partidas com PGN, cores, resultado e abertura	Chess.com: requisições em série, com User-Agent identificado e cache local
`buscar_puzzles`	`temas`, `rating_min`, `rating_max`, `quantidade` (padrão 5)	Puzzles com FEN, solução, rating e link	Consulta o recorte em SQLite
`buscar_conceitos`	`pergunta`, `k` (padrão 4)	Trechos das fichas com título e tema	Busca no ChromaDB

Exemplo de ferramenta com FastMCP

```
from fastmcp import FastMCP
import chess

mcp = FastMCP("lance-a-lance")

@mcp.tool()
def validar_lances(fen: str, lances: list[str]) -> list[dict]:
    """Verifica se cada lance é legal na posição dada."""
    resultado = []
    for lance in lances:
        board = chess.Board(fen)
```

```
        try:
            move = board.parse_san(lance)
            resultado.append({"lance": lance, "legal": True,
                              "san": board.san(move)})
        except ValueError:
            resultado.append({"lance": lance, "legal": False})
    return resultado

if __name__ == "__main__":
    mcp.run()
```

Pronto quando: todas as ferramentas têm testes com `pytest`, rodam pelo inspetor do FastMCP (`fastmcp dev server.py`) e respondem em menos de 2 segundos, exceto `analisar_partida`.

8. Componente 2 – Agente e RAG

O agente trabalha em dois modos: um fluxo fixo para explicar erros e um agente livre (estilo ReAct) para o chat. Nos dois, nenhum texto chega ao usuário sem passar pelo validador.

Classificação dos erros

A avaliação do motor (em centipeões, *cp*, do ponto de vista do jogador) vira uma chance de vitória de 0 a 100%. Usamos a mesma ideia de curva logística adotada pela Lichess:

```
\text{vitória}(cp) = 50 + 50 \cdot \left( \frac{2}{1 + e^{-0{,}00368208 \cdot cp}} - 1 \right)
```

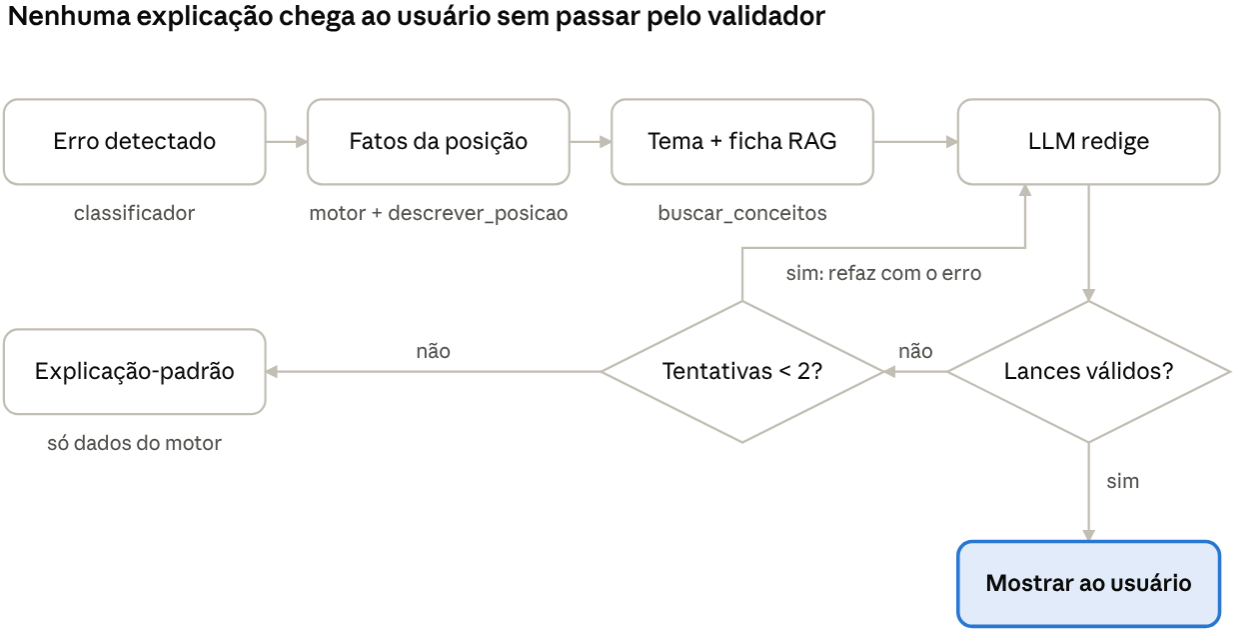
A queda Δ = vitória antes − vitória depois do lance define a classe:

Queda na chance de vitória	Classe
Δ ⩾ 30 pontos	Erro grave
20 ⩽ Δ < 30	Erro
10 ⩽ Δ < 20	Imprecisão
Δ < 10	Lance aceitável

Os limites ficam em um arquivo de configuração e têm testes unitários. Mate forçado é tratado como 100% ou 0%.

Modo 1 – Explicar um erro (fluxo fixo)

1. Recebe a posição antes do erro, o lance jogado, o melhor lance e a linha do motor.
2. Chama `descrever_posicao` antes e depois do lance para obter os fatos (peça que ficou indefesa, cravada, xeque descoberto...).
3. Pede ao LLM o tema tático, escolhido de uma lista fechada (garfo, cravada, espeto, ataque descoberto, peça pendurada, mate, sobrecarga, desvio, atração, outro).
4. Chama `buscar_conceitos` com o tema e recupera a ficha correspondente.
5. O LLM redige a explicação: o que aconteceu, por que é ruim, o que deveria ter sido jogado e a lição geral.
6. O validador extrai todos os lances do texto e os confere com `validar_lances` e com a linha do motor.
7. Se algo falhar, o LLM recebe o erro e refaz (no máximo 2 tentativas). Na terceira falha, a tela mostra uma explicação-padrão montada só com os dados do motor.



fluxo do modo 1 · validação com até 2 novas tentativas

Se o texto citar um lance inválido, o LLM recebe o erro e refaz; depois de duas falhas, o usuário vê uma explicação montada só com os dados do motor.

Modo 2 – Chat sobre a posição

Agente LangGraph com as ferramentas MCP carregadas por `langchain-mcp-adapters`. A cada pergunta, ele recebe o FEN da posição na tela e decide quais ferramentas chamar. A resposta final passa pelo mesmo validador do modo 1.

Prompt de sistema (resumo)

- Você é um treinador de xadrez que explica em português simples, para jogadores de até 1800 de rating.
- Nunca calcule lances por conta própria: use apenas lances e avaliações vindos das ferramentas.
- Escreva lances em notação algébrica (SAN) e cite o tema tático pelo nome da lista.
- Se as ferramentas não sustentarem uma afirmação, não a faça.
- Responda em no máximo 4 frases para explicações de erro.

RAG

- **Documentos:** as fichas em `knowledge_base/`, uma por tema, com metadados (`tema`, `nivel`).
- **Divisão:** uma ficha = um trecho (as fichas são curtas); se passar de ~500 palavras, dividir por subtítulo.
- **Busca:** similaridade no ChromaDB com k = 4, filtrando pelo tema quando ele já é conhecido.
- **Uso:** a ficha recuperada entra no prompt e o título dela é mostrado na interface como fonte (RF09).

9. Componente 3 – Interface web e API

Três telas bastam para o MVP, e a de análise é a estrela da demo.

Telas

1. **Início:** campo para o usuário do Chess.com (com quantidade de partidas) ou caixa para colar PGN. Lista das partidas importadas com resultado, abertura e número de erros graves.
2. **Análise da partida:**
   - Tabuleiro (`react-chessboard`) com setas para o lance jogado (vermelha) e o melhor lance (verde).
   - Lista de lances com cor por classe: amarelo para imprecisão, laranja para erro, vermelho para erro grave. Clicar leva o tabuleiro àquela posição.
   - Gráfico da avaliação ao longo da partida, clicável.
   - Painel de explicação com tema, texto do agente e ficha de conceito usada.
   - Chat sobre a posição atual, com indicação de quais ferramentas o agente usou.
3. **Relatório do jogador:** temas em que mais erra (gráfico de barras), evolução entre partidas e puzzles recomendados com link para a Lichess.

Estados obrigatórios em todas as telas: carregando (análise pode levar até 1 minuto, então mostrar progresso lance a lance), vazio e erro com mensagem clara (usuário inexistente, PGN inválido, motor indisponível).

Endpoints do backend

Método e rota	Função
`POST /api/partidas/importar`	Busca as partidas de um usuário do Chess.com ou recebe um PGN
`POST /api/partidas/{id}/analisar`	Inicia a análise e devolve o id da tarefa
`GET /api/partidas/{id}`	Partida com lances, avaliações, classes e explicações prontas
`GET /api/tarefas/{id}`	Progresso da análise (para a barra de carregamento)
`POST /api/explicar`	Explicação de um lance específico (FEN + lance)
`POST /api/chat`	Pergunta sobre a posição atual; resposta com ferramentas usadas
`GET /api/relatorio/{usuario}`	Temas fracos somados e puzzles recomendados

A análise roda em segundo plano (`BackgroundTasks` do FastAPI) e a interface consulta o progresso a cada 2 segundos.

10. Avaliação

O experimento central mostra, com números, que ancorar o LLM nas ferramentas elimina lances inventados e melhora as explicações. É o gráfico principal do README e do pitch.

Conjunto de teste (100 posições)

- **70 da base de puzzles da Lichess:** em cada puzzle, o primeiro lance da solução é o erro do adversário. Isso dá um erro real com tema já rotulado. Sortear com rating entre 800 e 2000, equilibrando os 8 temas principais.
- **30 de partidas reais** dos integrantes e de colegas, com o tema rotulado pelo grupo.
- Formato em `evaluation/dataset.jsonl` : `fen` , `lance_jogado` , `melhor_lance` , `tema_esperado` , `origem` .

Três versões comparadas

Versão	O que o LLM recebe
A – LLM puro	Só o FEN e o lance jogado
B – LLM + motor no prompt	FEN, lance jogado e saída do Stockfish colada no prompt, sem ferramentas nem validador
C – Agente completo	Fluxo do modo 1: ferramentas MCP, RAG e validador

Métricas

Métrica	Como medir	Meta para a versão C
Lances inválidos citados	% de explicações com ao menos um lance ilegal ou inexistente, checado por script	0%
Acerto do melhor lance	% em que o lance recomendado bate com o do Stockfish	⩾ 95%
Acerto do tema	% em que o tema bate com o rótulo da base	⩾ 60%
Qualidade percebida	Nota de 1 a 5 por 3 colegas, em amostra cega de 30 explicações por versão	⩾ 4,0
Fidelidade do RAG	Métrica de fidelidade (faithfulness) do RAGAS	⩾ 0,8
Custo e tempo	Tokens e segundos por explicação	Registrar e comparar

Como apresentar: `run_eval.py` roda as três versões e salva um CSV em `evaluation/results/`. O README mostra uma tabela com as métricas por versão, um gráfico de barras comparando A, B e C e três exemplos lado a lado de explicações da mesma posição. As metas são referências: se não forem atingidas, o resultado real vai para o README com uma análise do porquê.

11. Divisão do trabalho

Milton fica com toda a parte visual, que pesa no critério de usabilidade da interface. Os outros dois dividem o backend: [Integrante 1] cuida das ferramentas, dos dados e da API; [Integrante 2] cuida do agente, do RAG e da avaliação. Troquem os nomes entre colchetes.

Integrante	Responsável por	Entrega principal
Milton	Interface web, identidade visual, usabilidade e visual do pitch	3 telas prontas, teste de usabilidade (SUS), GIF e prints do README
[Integrante 1]	Servidor MCP, dados do Chess.com e dos puzzles, API FastAPI, cache e Docker	6 ferramentas testadas e o projeto subindo com `docker compose up`
[Integrante 2]	Agente, base de conceitos, RAG e avaliação	Explicações validadas e resultados A × B × C no README

Para o Milton não ficar travado: no dia 1 o grupo fecha o formato JSON de cada endpoint e gera arquivos de exemplo. Milton constrói as telas com esses dados falsos (mocks) e só troca pela API real na integração.

Todos (5/11 a 8/11)

- [ ] Validar a proposta com o professor
- [ ] Criar o repositório, a estrutura de pastas e o `.env.example`
- [ ] Escolher o provedor de LLM e conseguir a chave de API
- [ ] Fechar o contrato JSON dos endpoints e gerar os mocks para o front
- [ ] Rodar um "olá mundo" de cada parte (Stockfish, FastMCP, LangGraph, React)

Milton – Interface e experiência

Até 8/11

- [ ] Protótipo das 3 telas (Figma ou papel), aprovado pelo grupo
- [ ] Identidade visual: logo simples, paleta, tipografia e cores das classes de lance

Até 16/11

- [ ] Projeto React + Vite com rotas e layout base
- [ ] Tela de início: busca por usuário do Chess.com, lista de partidas e opção de colar PGN
- [ ] Tela de análise: tabuleiro com setas, lista de lances colorida, gráfico de avaliação clicável e painel de explicação com a ficha de conceito
- [ ] Chat da posição, mostrando quais ferramentas o agente usou
- [ ] Tela de relatório: gráfico de temas fracos e cartões de puzzles
- [ ] Estados de carregamento (progresso lance a lance), vazio e erro
- [ ] Layout responsivo (notebook e celular) e modo escuro

- [ ] Acessibilidade básica: contraste, navegação por teclado e classes de lance indicadas por ícone ou texto, não só por cor

Até 18/11

- [ ] Trocar os mocks pela API real, junto com [Integrante 1]

Até 21/11

- [ ] Teste de usabilidade com 5 colegas usando o questionário SUS (System Usability Scale, nota de 0 a 100); ajustes e resultado no README
- [ ] GIF da demo, prints das telas e visual do README
- [ ] Slides do pitch

[Marcelo] – Ferramentas, dados e API

Até 11/11

- [ ] Cliente da API do Chess.com: arquivos mensais, User-Agent, requisições em série e cache
- [ ] Script da base de puzzles com recorte em SQLite e dicionário de temas inglês → português
- [ ] Ferramentas `analisar_posicao` e `analisar_partida`
- [ ] Ferramentas `validar_lances` e `descrever_posicao`
- [ ] Ferramentas `buscar_partidas` e `buscar_puzzles`
- [ ] Testes `pytest` de todas as ferramentas

Até 16/11

- [ ] Endpoints FastAPI, análise em segundo plano e rota de progresso
- [ ] Cache em SQLite
- [ ] `docker-compose.yml` subindo tudo com um comando

Até 18/11

- [ ] Integração com o front (Milton) e com o agente ([Integrante 2])
- [ ] Seção "Como executar" do README

[Zanon] – Agente, RAG e avaliação

Até 15/11

- [ ] Escrever as ~30 fichas da base de conceitos (dividir com [Integrante 1] se atrasar)
- [ ] Indexação no ChromaDB e ferramenta `buscar_conceitos`
- [ ] Classificador de erros com testes

- [ ] Fluxo do modo 1 em LangGraph, com validador e até 2 novas tentativas
- [ ] Agente do modo 2 (chat) com as ferramentas MCP
- [ ] Prompts finais em `agent/prompts/`

Até 21/11

- [ ] Montar o `dataset.jsonl` de 100 posições
- [ ] `run_eval.py` com as versões A, B e C, tabela e gráfico
- [ ] Avaliação cega com 3 colegas
- [ ] Seções de arquitetura e resultados do README

Todos (18/11 a 22/11)

- [ ] Teste completo em uma máquina limpa, seguindo só o README
- [ ] Escolher e testar a partida da demo ao vivo
- [ ] Dois ensaios cronometrados do pitch

12. Cronograma

O MVP integrado fica pronto em 18/11; os cinco dias seguintes são reservados para avaliação, README e ensaio. Se algo atrasar, o corte é nos requisitos desejáveis, nunca nesses cinco dias.

MVP pronto em 18/11, cinco dias antes da entrega

Cronograma de novembro de 2026; marcas nas segundas-feiras



cronograma · 5/11 a 23/11

As três frentes correm em paralelo a partir de 9/11, uma por integrante; uma revisão conjunta em 12/11 confirma se a integração de 16/11 está no rumo.

13. Entregáveis

O enunciado exige quatro coisas no repositório: README explicativo, código organizado, instruções de reprodução e exemplos de resultados. O README abaixo cobre todas.

Estrutura do README.md

- [ ] **Título e frase de impacto**, com GIF da demo logo no topo
- [ ] **Problema:** a dor do jogador que não entende os próprios erros
- [ ] **Solução e abordagem:** agente + MCP + RAG e o princípio de grounding
- [ ] **Arquitetura:** diagrama (o mesmo da seção 3) e descrição de cada componente
- [ ] **Ferramentas MCP:** tabela da seção 7
- [ ] **Como executar:** pré-requisitos (Docker ou Python 3.11 + Node), clonar, copiar `.env.example`, baixar os dados com o script, `docker compose up`, endereço local para abrir
- [ ] **Como reproduzir a avaliação:** comando do `run_eval.py` e onde ficam os resultados
- [ ] **Resultados:** tabela de métricas A × B × C, gráfico e exemplos de explicações
- [ ] **Prints:** as três telas e um log de chamadas de ferramentas do agente
- [ ] **Limitações e próximos passos**
- [ ] **Equipe:** integrantes e o que cada um fez
- [ ] **Licenças e créditos:** base de puzzles da Lichess (CC0), dados públicos do Chess.com, Stockfish, bibliotecas usadas

Repositório

- [ ] Commits de todos os integrantes, distribuídos ao longo das semanas (não só no último dia)
- [ ] Nenhuma chave de API ou arquivo grande commitado
- [ ] `requirements.txt` e `package-lock.json` com versões fixas
- [ ] Testes passando com `pytest`

14. Roteiro do pitch (7 minutos)

O pitch segue os quatro pontos do enunciado, e a demo ao vivo ocupa quase metade do tempo. Os três integrantes falam.

Tempo	Conteúdo	Quem
0:00–1:00	**Problema:** mostrar o Stockfish dizendo "+2,3, melhor lance Cf5" e perguntar à turma se isso ensina algo	Milton
1:00–2:00	**Solução:** o treinador que explica; o diagrama de arquitetura e a regra "o LLM só explica, as ferramentas decidem"	[Integrante 1]
2:00–5:00	**Demo ao vivo:** importar a partida de um colega, abrir um erro grave, ler a explicação, fazer uma pergunta no chat e mostrar o relatório com puzzles	Milton conduz, [Integrante 2] narra
5:00–6:15	**Resultados:** gráfico A × B × C, com destaque para lances inventados (LLM puro contra agente)	[Integrante 2]
6:15–7:00	**Impacto e próximos passos:** comentário ao vivo durante a partida, voz, leitura do tabuleiro por foto, uso em escolinhas de xadrez	[Integrante 1]

Plano B da demo: gravar um vídeo da demo completa na véspera e ter as análises da partida escolhida já em cache. Se a internet ou a API falharem, o app continua mostrando o resultado salvo.

15. Riscos e plano B

O maior risco é o prazo; por isso cada risco abaixo tem uma saída que não depende de trabalho extra.

Risco	Prevenção	Plano B
Atraso na integração	Contrato JSON combinado no dia 1; integrar um fluxo simples já em 12/11	Cortar RF09–RF11 e o modo chat avançado
Custo ou limite da API do LLM	Cache de explicações; limitar a 10 partidas por usuário	Modelo local via Ollama
Stockfish lento	Limite de tempo por posição; analisar só os lances do jogador	Reduzir a profundidade para a demo

Risco	Prevenção	Plano B
LLM erra o tema tático	Lista fechada de temas + fatos de `descrever_posicao` no prompt	Mostrar só "melhor lance e por quê", sem tema
API do Chess.com fora do ar, com limite ou sem as partidas mais recentes	Cache das partidas importadas	Colar PGN manualmente (RF02)
Integrante sem commits suficientes	Componentes com dono e PRs revisados por outro	Redistribuir tarefas na revisão de 12/11
Demo falha ao vivo	Partida da demo analisada e em cache	Vídeo gravado da demo

16. Checklist final (até 22/11)

- [ ] Projeto clonado e executado do zero em outra máquina, seguindo só o README
- [ ] Todos os requisitos essenciais (RF01–RF08) funcionando
- [ ] Nenhum lance inválido nas explicações da versão C no conjunto de teste
- [ ] Resultados da avaliação, gráfico e exemplos no README
- [ ] Prints das três telas e GIF da demo no README
- [ ] Commits de todos os integrantes visíveis no GitHub
- [ ] Partida da demo analisada e em cache; vídeo de backup gravado
- [ ] Pitch ensaiado em até 7 minutos, com fala dos três
- [ ] Link do repositório entregue antes de `Nov 23, 2026` às 12h00

17. Fontes

- Chess.com Published-Data API – referência oficial
- Guia de uso da API do Chess.com: endpoints, arquivos mensais e User-Agent
- Lichess open database – puzzles e licença CC0
- python-chess – comunicação com motores UCI
- FastMCP – documentação
- FastMCP no PyPI
- langchain-mcp-adapters